

昇腾架构学习与实践报告



——基于Ascend 910B的计算机视觉模型迁移与训练 (<https://github.com/YaoJiaming-cn/Ascend>)

1. 昇腾全栈架构理解

昇腾NPU的架构可分为四层，与NVIDIA GPU架构对比关系如下：

层次	NVIDIA 生态	昇腾生态	作用
硬件	NVIDIA GPU	达芬奇架构 (Ascend 910B)	物理算力芯片
计算架构	CUDA + cuDNN	CANN	算子
框架适配	torch.cuda (PyTorch内置)	torch_npu (独立插件)	中间层
框架	PyTorch / TensorFlow	PyTorch/ MindSpore	模型开发接口

关键理解：

- 昇腾的架构中，为了适配pytorch框架，使pytorch能调用NPU硬件，加入了独立的 `torch_npu` 插件作为中间层才得以实现，而pytorch对CUDA的支持是原生的（`torch.cuda` 内置）。在代码层面，迁移的核心变更仅为 `import torch_npu` 和使用 `.to('npu:0')` 替代 `.to('cuda:0')`。
- 另外，昇腾 910B 的FP16峰值算力为320 TFLOPS，与NVIDIA A100（312 TFLOPS）处于同一量级（大概有80%）。本文所有实验均在华为云ModelArts平台搭载的单卡Ascend 910B4实例上完成。

2. 开发环境搭建与踩坑记录

1. 环境选型过程

最初尝试了AtomGit AI提供的免费昇腾Notebook（云端Ascend 910B4，限时2小时），但在训练阶段频繁遇到共享内存不足（Bus error）和kernel自动重启的问题，稳定性无法满足完整训练需求。后续转向华为云ModelArts付费实例（单卡snt9b1，约18.68元/小时），使用预置镜像 `pytorch_2.7.1-cann_8.5.2`，稳定性很强，大大降低了环境配置成本。

2. 关键问题与解决方案

问题	原因	解决方案
DataLoader Bus error	免费容器共享内存 (/dev/shm) 不足	切换到付费实例，增大 num_workers
CIFAR-10下载极慢 (~10KB/s)	pytorch.org 服务器在海外	通过VPN加拿大节点下载后上传
CANN SyntaxWarning刷屏	CANN工具链部分Python代码使用了旧式字符串转义	设置sys.warnoptions屏蔽
ultralytics check_amp() 崩溃	调用 torch.cuda.get_device_name(), NPU 环境无 CUDA	设置 `amp=False` 跳过混合精度检查
torchvision::nms 不支持 NPU	CANN 算子库未覆盖 NMS 算子	自动 CPU fallback，后处理耗时增加
model.val() 回退 CPU	训练结束后模型默认回到 CPU，val() 未指定 device	显式传 device='npu:0'
_clear_memory() 崩溃	调用 torch.cuda.get_device_properties() 检查显存	monkey-patch 该函数，NPU 设备直接 return

3. ResNet-50 在昇腾NPU上的训练实验

1. 实验环境与设置

配置项	值
硬件	1 × Ascend 910B4 (29.5 GB HBM)
CPU架构	aarch64 (鲲鹏920)
云平台	华为云ModelArts (西南-贵阳)
CANN版本	8.5.2
数据集	CIFAR-10 (50k训练 / 10k测试)
优化器	SGD, lr=0.05, momentum=0.9
学习率策略	CosineAnnealingLR (T_max=15)
损失函数	CrossEntropyLoss
Batch size	128
训练轮数	15 epochs
Python	3.12.0
PyTorch	2.7.1+cpu
torch_npu	2.7.1.post2

2. 训练结果

阶段	训练集准确率	验证集准确率
Epoch 1	16.80%	23.80%
Epoch 5	61.30%	62.40%
Epoch 10	82.50%	79.20%
Epoch 15	95.60%	84.3%（最佳）

如表格所示，最终epoch为15时模型效果达到最佳，训练集与验证集准确率存在约11个百分点的差距（95.6% vs 84.3%），这表明模型存在一定程度的过拟合。总训练时间：43.3分钟（2599秒），平均每epoch约173秒。训练/验证 loss 与 accuracy 随epoch变化曲线如图1所示。

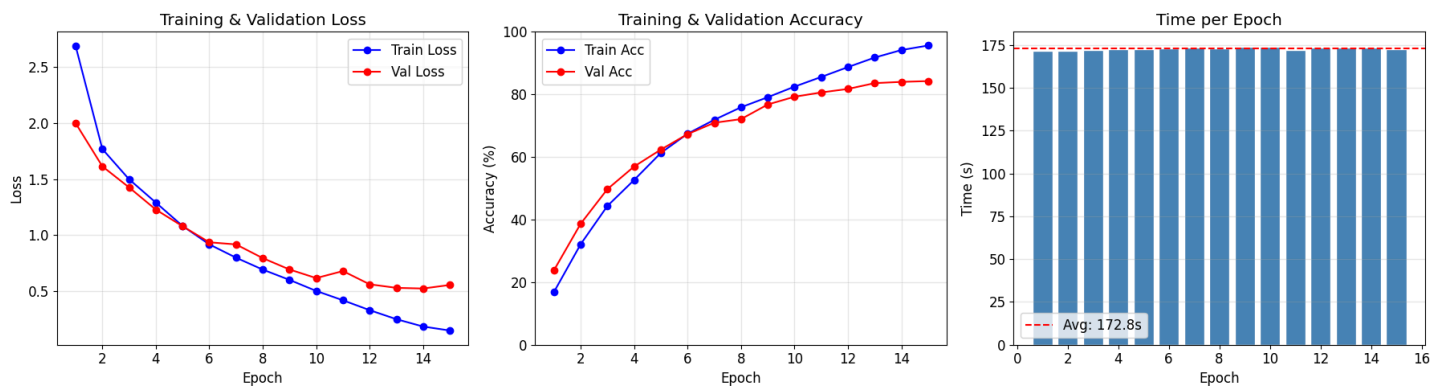


图1 训练/验证 loss 与 accuracy 随epoch变化曲线图

3. 性能对比分析

设备	单batch训练耗时	加速比
CPU（鲲鹏920 ARM）	53.5s	1×
NPU（Ascend 910B4）	0.42s	127.6×

如表格所示，NPU在训练加速方面表现优异（127倍于CPU）。但需要注意，当前实验使用FP32全精度训练，未启用混合精度（FP16）和更大的batch size。参考同类实验数据，若开启AMP自动混合精度并将batch size增大至256，预期epoch耗时可从173s降至约90-100s，进一步接近A100在同等配置下的性能水平（约60-80s/epoch）。

4. YOLOv8 目标检测在昇腾NPU上的迁移与训练

1. 实验目的

在完成 ResNet50 图像分类任务后，为了进一步感受昇腾 NPU 在目标检测领域的表现，并熟悉昇腾架构和训练框架在昇腾上的兼容性，于是在910B上进行YOLOv8的实验，本实验的核心目的：

- 验证 ultralytics 训练框架在昇腾 NPU 上的兼容性

- 完成完整的 训练（基于yolov8n.pt进行Fine-tuning）→ 验证 → 推理 流程
- 记录 NPU 迁移中的特殊踩坑（详情见2.2关键问题）

2. 实验设置

配置项	值
模型	YOLOv8n（3.15M 参数，8.9 GFLOPs）
预训练权重	yolov8n.pt（COCO 预训练）
数据集	COCO128（128张图，80类）
训练轮数	10 epochs
图片尺寸	320×320
Batch size	8
优化器	AdamW（auto）
混合精度	关闭（AMP 不兼容 NPU）
DataLoader workers	4

3. 训练过程与结果

将代码迁移到云端 NPU 后，并非简单的 `cuda:0` → `npu:0` 就能跑通。ultralytics 内部有两处硬编码了 `torch.cuda` 调用（详情见2.2关键问题）

训练过程 10 个 epoch 总耗时约 2.2 分钟，各 epoch 指标变化如下：

Epoch	box_loss	cls_loss	dfl_loss	mAP@50	mAP@50-95
1	1.344	1.868	1.207	0.525	0.388
3	1.234	1.587	1.136	0.543	0.396
5	1.237	1.488	1.153	0.569	0.412
7	1.245	1.438	1.153	0.569	0.417
10	1.239	1.294	1.154	0.59	0.435

训练损失稳步下降，mAP 持续提升，训练曲线图如图2所示。最终 mAP@50=0.590，mAP@50-95=0.435。考虑到 coco128 仅 128 张图且只训练 10 轮，这个结果合理——预训练权重已经提供了强大的基础特征提取能力，微调主要是在适应数据分布。



图4 验证集检测结果示例 (模型预测框, 置信度阈值 0.5)

4. NPU 推理性能测试

加载训练产出的 best.pt，对 test_bus.jpg 进行 50 次推理计时（含 5 次预热）：

指标	数值
单张端到端推理	26.2 ms
FPS	38.1
纯 NPU 前向推理	0.6 ms/张（训练时 val）

NPU中端到端的 26.2ms 中，大部分消耗在 CPU 侧的 NMS 后处理上。纯 NPU 推理仅需 0.6ms，说明 Ascend 910B4 的算力对于 YOLOv8n 这种轻量模型完全足够。

如图5所示，模型准确检出了图中的公交车（bus，置信度~0.87）和多名行人（person），检测框位置与物体轮廓基本吻合。



图5 NPU 端到端推理测试 (26.2ms)

5. 实验总结

与 ResNet-50 实验对比的话，ResNet 分类任务（torchvision 原生模型 + 手写训练循环）在 NPU 上能直接适应，0 迁移成本，仅需 `import torch_npu + .to('npu:0')`。而 YOLOv8 依赖第三方框架 ultralytics，框架内部的 CUDA 假设成为迁移的主要障碍。反映了当前昇腾架构的困境：因为算子覆盖尚未达到 CUDA 的完备程度，越接近“原生 PyTorch”的代码迁移越顺畅，而依赖复杂第三方框架时可能需要 debug。